```
// g++ -O2 -std=c++17 lemma22_verify_exact.cpp -lgmp -o lemma22_verify
// ./lemma22_verify > lemma22_verification_output.txt
// Lemma 2.2: d1(t)=p(t)-p(t-1), with p(t)=0 for t<0.
// Verify D_d1(t)=d1(t)^2-d1(t-1)*d1(t+1) for 72<=t<10000.
// All mathematical calculations use exact GMP integers or rationals.
// The large-range analytic argument remains the one in the paper.

#include <gmp.h>
#include <algorithm>
#include <chrono>
#include <iostream>
#include <vector>

struct Integer
{
    mpz_t value;

    Integer()
    {
        mpz_init(value);
    }

    ~Integer()
    {
        mpz_clear(value);
    }

    Integer(const Integer &) = delete;
    Integer &operator=(const Integer &) = delete;
};

int checks = 0, failures = 0;

void check(bool condition, const char *description)
{
    ++checks;
    if (!condition)
        ++failures;
    std::cout << (condition ? "PASS " : "FAIL ") << description << '\n';
}

void positive_taylor_sum(mpq_t sum, unsigned long numerator,
                         unsigned long denominator, unsigned degree)
{
    mpq_t x, term;
    mpq_inits(x, term, nullptr);
    mpq_set_ui(x, numerator, denominator);
    mpq_canonicalize(x);
    mpq_set_ui(sum, 1, 1);
    mpq_set_ui(term, 1, 1);
    for (unsigned k = 1; k <= degree; ++k)
    {
        mpq_mul(term, term, x);
        mpz_mul_ui(mpq_denref(term), mpq_denref(term), k);
        mpq_canonicalize(term);
        mpq_add(sum, sum, term);
    }
    mpq_clears(x, term, nullptr);
}

int main()
{
    const auto start = std::chrono::steady_clock::now();
    constexpr int first = 72, last = 9999, limit = last + 1;
    std::vector<Integer> p(limit + 1), d1(limit + 1), no_ones(limit + 1);

    // Euler's pentagonal recurrence, including the required neighbor 10000.
    mpz_set_ui(p[0].value, 1);
    for (int n = 1; n <= limit; ++n)
    {
        for (int r = 1;; ++r)
```

```
    {
        const int a = r * (3 * r - 1) / 2;
        if (a > n)
            break;
        const int b = a + r;
        if (r & 1)
        {
            mpz_add(p[n].value, p[n].value, p[n - a].value);
            if (b <= n)
                mpz_add(p[n].value, p[n].value, p[n - b].value);
        }
        else
        {
            mpz_sub(p[n].value, p[n].value, p[n - a].value);
            if (b <= n)
                mpz_sub(p[n].value, p[n].value, p[n - b].value);
        }
    }
}
mpz_set_ui(d1[0].value, 1);
for (int t = 1; t <= limit; ++t)
    mpz_sub(d1[t].value, p[t].value, p[t - 1].value);
std::cout << "PARTITIONS_READY first=0 last=" << limit
          << " count=" << limit + 1 << " arithmetic=GMP_exact_integer\n";

// Independently compute coefficients of product_{k>=2}(1-q^k)^(-1).
mpz_set_ui(no_ones[0].value, 1);
for (int part = 2; part <= limit; ++part)
    for (int n = part; n <= limit; ++n)
        mpz_add(no_ones[n].value, no_ones[n].value,
                no_ones[n - part].value);
int mismatches = 0, nonpositive = 0, decreases = 0;
for (int t = 0; t <= limit; ++t)
    if (mpz_cmp(d1[t].value, no_ones[t].value) != 0)
        ++mismatches;
for (int t = 2; t <= limit; ++t)
{
    if (mpz_sgn(d1[t].value) <= 0)
        ++nonpositive;
    if (t < limit && mpz_cmp(d1[t].value, d1[t + 1].value) > 0)
        ++decreases;
}
check(mismatches == 0, "independent_no_ones_product n=0..10000 coefficients=10001");
check(nonpositive == 0, "d1_positive t=2..10000");
check(decreases == 0, "d1_nondecreasing t=2..10000 comparisons=9998");
if (mismatches != 0)
    return 1;

mpz_t determinant, product, minimum;
mpz_inits(determinant, product, minimum, nullptr);
bool have_minimum = false;
std::vector<int> minimizers;
int compared = 0, negatives = 0, zeros = 0;
for (int lo = first; lo <= last; lo += 1000)
{
    const int hi = std::min(last, lo + 999);
    int block_count = 0, block_negatives = 0, block_zeros = 0;
    for (int t = lo; t <= hi; ++t)
    {
        mpz_mul(determinant, d1[t].value, d1[t].value);
        mpz_mul(product, d1[t - 1].value, d1[t + 1].value);
        mpz_sub(determinant, determinant, product);
        const int sign = mpz_sgn(determinant);
        if (sign < 0)
            ++block_negatives;
        if (sign == 0)
            ++block_zeros;
        if (!have_minimum || mpz_cmp(determinant, minimum) < 0)
        {
            mpz_set(minimum, determinant);
            minimizers.assign(1, t);
```

```
                have_minimum = true;
            }
            else if (mpz_cmp(determinant, minimum) == 0)
                minimizers.push_back(t);
            ++block_count;
        }
        compared += block_count;
        negatives += block_negatives;
        zeros += block_zeros;
        std::cout << (block_negatives ? "BLOCK_FAIL" : "BLOCK_PASS")
                  << " t=" << lo << ".." << hi << " checked=" << block_count
                  << " negatives=" << block_negatives << " zeros=" << block_zeros
                  << " cumulative=" << compared << '\n';
    }
    check(compared == last - first + 1, "coverage t=72..9999 cases=9928");
    check(negatives == 0, "D_d1(t)>=0 at every t=72..9999");
    check(zeros == 0, "D_d1(t)>0 at every t=72..9999");
    check(mpz_cmp_ui(minimum, 28877078UL) == 0,
          "minimum_D_d1_equals_28877078");
    check(minimizers.size() == 1 && minimizers[0] == 73,
          "unique_minimizer_t_equals_73");
    gmp_printf("MINIMUM D_d1=%Zd\n", minimum);
    std::cout << "MINIMIZERS";
    for (int t : minimizers)
        std::cout << " t=" << t;
    std::cout << '\n';

    // Exact endpoint checks in the displayed large-range argument.
    check(99 * 99 < 9998, "sqrt(9998)>99");
    check(5 * 100 > 4 * 3,
          "5/(4*sqrt(t))-3/t>0 for t>=10000 by sqrt(t)>=100");
    mpq_t sum, bound;
    mpq_inits(sum, bound, nullptr);
    mpz_t integer_bound;
    mpz_init(integer_bound);
    positive_taylor_sum(sum, 495, 2, 10);
    mpz_ui_pow_ui(integer_bound, 10000, 3);
    mpz_mul_ui(integer_bound, integer_bound, 3072);
    mpq_set_z(bound, integer_bound);
    check(mpq_cmp(sum, bound) > 0, "S_10(495/2)>3072*10000^3");
    gmp_printf("TAYLOR S_10(495/2)=%Qd\n", sum);
    gmp_printf("TAYLOR required_bound=%Zd\n", integer_bound);

    // The preceding part of the same lemma uses this second Taylor bound.
    positive_taylor_sum(sum, 50, 1, 50);
    mpz_ui_pow_ui(integer_bound, 10, 20);
    mpz_mul_ui(integer_bound, integer_bound, 6);
    mpq_set_z(bound, integer_bound);
    check(mpq_cmp(sum, bound) > 0, "S_50(50)>6*10^20");

    mpq_clears(sum, bound, nullptr);
    mpz_clears(determinant, product, minimum, integer_bound, nullptr);
    const auto elapsed = std::chrono::duration_cast<std::chrono::microseconds>(
                             std::chrono::steady_clock::now() - start)
                             .count();
    std::cout << (failures ? "COMPLETE_FAIL" : "COMPLETE_PASS")
              << " finite_cases=" << compared << " checks=" << checks
              << " failed=" << failures << " negatives=" << negatives
              << " zeros=" << zeros << " elapsed_us=" << elapsed << '\n';
    return failures ? 1 : 0;
}
```